

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC VĂN HIẾN



TIỂU LUẬN KẾT THÚC HỌC PHẦN

MÔN: Bảo mật trong IoT

Mô phỏng phòng thủ MQTT trong môi trường lab

SVTH: Lê Hữu Lý

MSSV: 231A011195

Lớp học phần: 253INT441001

Giảng viên hướng dẫn: Hồ Nhựt Minh

Năm học: 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC VĂN HIẾN
KHOA CÔNG NGHỆ THÔNG TIN**



**TIỂU LUẬN KẾT THÚC HỌC PHẦN
BẢO MẬT IoT**

MÔ PHỎNG PHÒNG THỦ MQTT TRONG MÔI TRƯỜNG LAB

**Giảng viên hướng dẫn: Hồ Nhật Minh
Lớp học phần: 253INT441001
Sinh viên thực hiện: Lê Hữu Lý
Mã số sinh viên: 231A011195**

TP. HỒ CHÍ MINH

BẢN THẢO TIỂU LUẬN TÍCH LŨY
BÁO CÁO CUỐI KỲ — BUỔI 06/06 (BẢN HOÀN CHỈNH)
HỌC PHẦN: BẢO MẬT IoT (INT4410)

Mô phỏng phòng thủ MQTT trong môi trường lab

Mã đề tài	49
Hướng đề tài	B
Sinh viên	Lê Hữu Lý
Mã số sinh viên	231A011195
Lớp học phần	INT4410
Giảng viên	Hồ Nhật Minh
Link repo GitHub	https://github.com/huuly721-design/mqtt-defense-lab
Ngày nộp bản cuối kỳ	3/8/2026

TP. HỒ CHÍ MINH, 2026

LỜI CAM ĐOAN

Em tên là: Lê Hữu Lý, Mã số sinh viên: 231A011195.

Em xin cam đoan tiểu luận học phần Bảo mật IoT với đề tài "**Mô phỏng phòng thủ MQTT trong môi trường lab**" là công trình do chính em tự nghiên cứu, cấu hình và thực hiện dưới sự hướng dẫn chuyên môn của Thầy Hồ Nhật Minh.

Toàn bộ kết quả thực nghiệm, mã nguồn mô phỏng (Python scripts), cấu hình hệ thống (Mosquitto Broker, ACL, chứng chỉ TLS) và các hình ảnh log minh chứng được trình bày trong báo cáo là hoàn toàn trung thực, được trích xuất trực tiếp từ môi trường lab cục bộ (localhost) do em tự thiết lập.

Các tài liệu tham khảo, lý thuyết nền tảng và công cụ sử dụng (Eclipse Mosquitto, Paho MQTT, OpenSSL) đều được trích dẫn nguồn gốc rõ ràng, minh bạch, tuyệt đối tuân thủ các quy định về an toàn và đạo đức học thuật của Khoa Công nghệ Thông tin và nhà trường. Đề tài không chứa các dữ liệu cá nhân thật, mật khẩu hay tài nguyên nhạy cảm của bất kỳ tổ chức, cá nhân nào.

Nếu có bất kỳ sự gian lận hay vi phạm quy chế nào, em xin hoàn toàn chịu trách nhiệm trước Bộ môn, Khoa và Ban Giám hiệu nhà trường.

TP.Hồ Chí Minh, ngày 26 tháng 07 năm 2026
Sinh viên thực hiện

Lê Hữu Lý

PHIẾU XÁC ĐỊNH YÊU CẦU RIÊNG CỦA ĐỀ TÀI (ĐỀ TÀI 49)

STT	Mục tiêu cần đạt (Diễn giải thành mục tiêu đo được)	Sản phẩm / đầu ra bắt buộc
MT-01	Dựng lab MQTT phòng thủ có xác thực danh tính, cấu hình phân quyền ACL và thiết lập thành công tùy chọn đường hầm mã hóa TLS.	• <code>mosquitto.conf</code>, <code>aclfile</code>, <code>passwordfile</code> (kèm hướng dẫn tạo).
MT-02	Chứng minh client đúng quyền hoạt động truyền/nhận thành công; client sai quyền (hoặc nặc danh) bị hệ thống Broker chặn đứng.	• 3 script Paho Python (Publisher, Subscriber, Unauthorized). • Ảnh/log các kịch bản test.
MT-03	Viết tài liệu README chi tiết để sinh viên/giảng viên khác có thể thiết lập và chạy lại chính xác môi trường lab.	• File <code>README.md</code> hướng dẫn chạy lại lab từng bước.

MỤC LỤC

CHƯƠNG 1: MỞ ĐẦU	1
1.1 Bối cảnh và lý do chọn đề tài	1
1.2 Phát biểu vấn đề cụ thể	1
1.3 Mục tiêu cụ thể.....	1
1.4 Đối tượng - Phạm vi - Giới hạn pháp lý.....	1
1.5 Sản phẩm dự kiến.....	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	4
2.1 Kiến thức nền THỰC SỰ dùng.....	4
2.2 Khái niệm bảo mật gắn ví dụ của đề tài	4
2.3 Bảng tài liệu/công cụ tham khảo chính	4
2.4 Công trình liên quan: So sánh và Kế thừa.....	5
CHƯƠNG 3: PHƯƠNG PHÁP VÀ THIẾT KẾ.....	6
3.1 Phương pháp nghiên cứu	6
3.2 Sơ đồ kiến trúc bảo mật	6
3.3 Bảng cấu hình thành phần công cụ	6
3.4 Bảng quy trình chi tiết.....	7
3.5 Giới hạn an toàn và Đạo đức nghề nghiệp	8
Chương 4: TRIỂN KHAI % SẢN PHẨM.....	9
4.1 Các bước cài đặt.....	9
4.2 Bảng thành phần sản phẩm	10
4.3 Bảng kịch bản kiểm thử (test case)	11
4.4 Hướng dẫn chạy và sử dụng hệ thống.....	12
CHƯƠNG 5: KẾT QUẢ VÀ THẢO LUẬN.....	14
5.2. Bảng tổng hợp đối chiếu kết quả kiểm thử	19
5.3. Đối chiếu từng mục tiêu ở Mục 1.3	20
5.4. Hạn chế trung thực.....	20
CHƯƠNG 6: ĐÁNH GIÁ BẢO MẬT	21
6.1 BẢNG tài sản/dữ liệu.....	21
6.2 Bảng phân tích mối đe dọa (Threat Analysis - T-01 đến T-06)	22
6.3 Ma trận rủi ro R-01...R-06	22
6.4 Ưu tiên biện pháp xử lý rủi ro.....	23

6.5 Đánh giá sau xử lý	23
CHƯƠNG 7: KẾT LUẬN	24
7.1. Tóm tắt vấn đề, cách làm, kết quả nổi bật và mức độ hoàn thành mục tiêu	24
7.2. Hướng phát triển thực tế	24
TÀI LIỆU THAM KHẢO	25

CHƯƠNG 1: MỞ ĐẦU

1.1 Bối cảnh và lý do chọn đề tài

Giao thức MQTT đang là tiêu chuẩn truyền tải dữ liệu xương sống cho các hệ thống IoT. Tuy nhiên, ở trạng thái mặc định, các MQTT Broker (điển hình như Eclipse Mosquitto) thường ưu tiên tốc độ và sự tiện lợi, dẫn đến việc thiết lập sẵn các cấu hình kém an toàn. Luồng dữ liệu mặc định không được bọc mã hóa, đồng thời cơ chế kiểm duyệt danh tính thiết bị bị bỏ ngỏ, tạo ra điểm yếu chí mạng ở cả tầng mạng lẫn tầng ứng dụng.

1.2 Phát biểu vấn đề cụ thể

Việc triển khai MQTT Broker với cấu hình gốc (mở cổng TCP 1883, cho phép kết nối ẩn danh thông qua `allow_anonymous true`, và truyền dữ liệu dạng bản rõ - plaintext) trực tiếp dẫn đến ba rủi ro bảo mật nghiêm trọng:

1. Kẻ tấn công dễ dàng sử dụng kỹ thuật Man-in-the-Middle (MitM) để bắt gói tin và đọc trộm dữ liệu (Sniffing).
2. Nguy cơ giả mạo thiết bị (Spoofing) để tiêm nhiễm dữ liệu sai lệch vào hệ thống.
3. Thiết bị vắng lai có thể tùy ý đăng ký (Subscribe) hoặc gửi lệnh (Publish) do không có cơ chế chặn quyền. Vấn đề cấp thiết đặt ra là phải tái cấu trúc Broker, thiết lập ngay lập tức cơ chế phòng thủ 3 lớp (Mã hóa, Xác thực và Phân quyền) để chặn đứng các kịch bản tấn công trên.

1.3 Mục tiêu cụ thể

Đề tài đặt ra 4 mục tiêu có thể đo lường (ĐO ĐƯỢC) nhằm giải quyết triệt để vấn đề:

- ❖ **MT-01:** Chuyển đổi 100% luồng giao tiếp MQTT sang kênh bảo mật TLS 1.2 (qua cổng 8883), đảm bảo gói tin hoàn toàn bị mã hóa, không lộ chuỗi JSON. (Gắn với: Bộ chứng chỉ số tự ký).
- ❖ **MT-02:** Vô hiệu hóa triệt để kết nối ẩn danh, thiết lập cơ chế định danh thành công cho 3 nhóm tài khoản thiết bị riêng biệt. (Gắn với: Tập cấu hình Broker và cơ sở dữ liệu mật khẩu).
- ❖ **MT-03:** Cấu hình Access Control List (ACL) giới hạn chính xác quyền Read/Write cho từng nhóm tài khoản (sensor, dashboard, controller) trên các topic cụ thể (iot/sensor/#, iot/control/#), ngăn chặn 100% hành vi leo thang đặc quyền. (Gắn với: Tập chính sách phân quyền ACL).
- ❖ **MT-04:** Mô phỏng thành công 3 kịch bản thiết bị (Hợp lệ, Bị giới hạn quyền, và Ẩn danh/Hacker) để kiểm thử mức độ phòng thủ, xuất được log từ chối truy cập (Access Denied) từ Broker. (Gắn với: Mã nguồn giả lập thiết bị và Log hệ thống).

1.4 Đối tượng - Phạm vi - Giới hạn pháp lý

- ❖ **Đối tượng nghiên cứu:** Cơ chế bảo mật và phòng thủ rủi ro mạng (Authentication, Authorization, TLS Encryption) trên giao thức MQTT.

- ❖ **Phạm vi triển khai:** Đề tài thực nghiệm cục bộ (Localhost) trên hệ điều hành Windows 11. Các công cụ sử dụng giới hạn gồm Eclipse Mosquitto Broker và thư viện Paho MQTT Python (phiên bản 1.6.1).
- ❖ **Giới hạn pháp lý và Đạo đức nghề nghiệp:**
 - Toàn bộ chứng chỉ số (CA, Server/Client Certificate) được sử dụng trong đề án là loại **tự ký (Self-signed)**, chỉ có giá trị mô phỏng trong không gian thực hành (Lab), hoàn toàn KHÔNG có giá trị xác thực danh tính trên Internet hay môi trường Production thực tế.
 - Các hành vi kiểm thử tấn công (bắt gói tin, giả mạo ản danh) chỉ được phép mô phỏng nội bộ trên chính hệ thống do sinh viên tự dựng, tuân thủ nghiêm ngặt nguyên tắc không xâm phạm hệ thống thực tế của bên thứ ba.

1.5 Sản phẩm dự kiến

Sản phẩm được thực hiện trên máy tính cục bộ, sẽ có các sản phẩm và đầu ra bao gồm:

STT Mục tiêu	Tên mục tiêu	Sản phẩm/Đầu ra tương ứng
MT-01	Mã hóa TLS toàn bộ luồng MQTT	Thư mục configs/: Chứa bộ khóa và chứng chỉ số tự ký (ca.crt, server.crt, server.key).
MT-02	Xác thực danh tính người dùng	Tệp configs/mosquitto.conf (đã tắt anonymous) và tệp cơ sở dữ liệu mã hóa configs/passwordfile.
MT-03	Phân quyền truy cập tối thiểu	Tệp chính sách configs/aclfile chứa luật Read/Write chuẩn xác cho 3 nhóm role.
MT-04	Kịch bản mô phỏng kiểm thử	Thư mục src/: 3 mã nguồn Python (publisher.py, subscriber.py, unauthorized_client.py) kèm hình ảnh Log bắt lỗi từ Broker

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Kiến thức nền THỰC SỰ dùng

- ❖ **Kiến trúc MQTT (Pub/Sub):** Sử dụng Eclipse Mosquitto làm Broker trung chuyển. Các thiết bị (Client Python) gửi/nhận dữ liệu gián tiếp.
- ❖ **Topic & Wildcard:** Dùng ký tự # phân cụm logic (vd: `iot/sensor/#` cho cảm biến, `iot/control/#` cho điều khiển).
- ❖ **Giao thức TLS/SSL:** Dùng OpenSSL tạo chứng chỉ tự ký (Self-signed), ép giao tiếp qua cổng bảo mật 8883.
- ❖ **Thư viện Paho MQTT (v1.6.1):** Lập trình Client bằng Python, dùng cấu hình `tls_set()` để bọc mã hóa gói tin.

2.2 Khái niệm bảo mật gắn ví dụ của đề tài

- ❖ **Xác thực danh tính (Authentication):** Broker ép `allow_anonymous` false. Thiết bị vắng lai bị từ chối ngay lập tức; chỉ thiết bị có tài khoản trong `passwordfile` mới được kết nối.
- ❖ **Phân quyền truy cập (Authorization - ACL):** Áp dụng quyền tối thiểu. Ví dụ: Dù tài khoản `sensor` đã đăng nhập, nhưng nếu cố gửi lệnh vào `iot/control/#`, Broker sẽ lập tức chặn (Denied PUBLISH) dựa trên luật của `aclfile`.
- ❖ **Mã hóa đường truyền và chống nghe lén (Encryption in Transit):** Dữ liệu JSON bị khóa bằng thuật toán TLS trước khi rời thiết bị. Nếu kẻ tấn công (hoặc Client vắng lai) cố tình dòm ngó, hệ thống Broker chỉ ghi nhận các luồng kết nối bị từ chối do không vượt qua được bước xác thực chứng chỉ số (Certificate) trên cổng 8883.

2.3 Bảng tài liệu/công cụ tham khảo chính

Đề tài được sử dụng các công cụ công nghệ thông tin , và các có các phần được sử dụng chính trong bài như sau:

STT	Nguồn / Repo / Tool	URL	Phần đã sử dụng trong đề tài	Ngày truy cập
1	Eclipse Mosquitto (Tool chính)	https://github.com/eclipse/mosquitto	Cài đặt Broker, tra cứu cú pháp cấu hình <code>mosquitto.conf</code> , lệnh tạo <code>passwordfile</code> và <code>aclfile</code> .	25/07/2026

2	Paho MQTT Python (Repo)	https://github.com/eclipse/paho.mqtt.python	Sử dụng thư viện v1.6.1 để viết script pub/sub, cấu hình hàm bảo mật TLS.	25/07/2026
3	OpenSSL (Tool)	https://github.com/openssl/openssl	Tham khảo bộ lệnh CLI tạo Root CA, Server Certificate và Private Key tự ký (Self-signed).	25/07/2026
4	MQTT Security Fundamentals	https://www.hivemq.com/mqtt-security-fundamentals	Tham khảo cơ sở lý thuyết về Zero-Trust, cách thiết lập ACL và nguyên lý hoạt động của MQTT over TLS.	

(bảng 2.1 bảng tài liệu / công cụ tham khảo)

2.4 Công trình liên quan: So sánh và Kế thừa

- ❖ **Công trình liên quan:** Các đồ án IoT cơ bản hoặc mã nguồn mở thường chỉ tập trung vào tính năng truyền tải. Chúng đa số sử dụng Broker cấu hình mặc định (mở cổng 1883, không mã hóa, cho phép kết nối ẩn danh) hoặc phụ thuộc hoàn toàn vào các nền tảng Cloud, bỏ qua việc tự quản trị an toàn thông tin tại hạ tầng lõi.

- ❖ **So sánh điểm khác biệt:**

- **Quản trị On-premise:** Triển khai Broker độc lập, cấu hình trực tiếp qua các tệp tin hệ thống thay vì phụ thuộc vào giao diện nền tảng (Black-box).
- **Phòng thủ chiều sâu:** Tích hợp chuỗi bảo mật 3 lớp chặt chẽ: Ép luồng mạng qua cổng 8883 (TLS) → Định danh (chặn Anonymous) → Phân quyền (ACL).
- **Kiểm chứng thực chiến:** Tự xây dựng kịch bản kiểm thử bằng cách chạy scripts (unauthorized_client.py) và kết hợp với việc theo dõi Log trực tiếp từ Broker (dưới chế độ giám sát -v) từ đó xuất được minh chứng thực tế từ hệ thống nhận diện và từ chối truy cập (Access Denied / Not Authorized)

- ❖ **Phản kế thừa:**

- Kế thừa nguyên lý **Zero-Trust** (Đặc quyền tối thiểu) để xây dựng bộ luật cho tệp ACL.
- Kế thừa kiến trúc định tuyến Topic từ tổ chức Eclipse Foundation và tiêu chuẩn hạ tầng khóa công khai (PKI) để sinh chứng chỉ số tự ký qua OpenSSL.

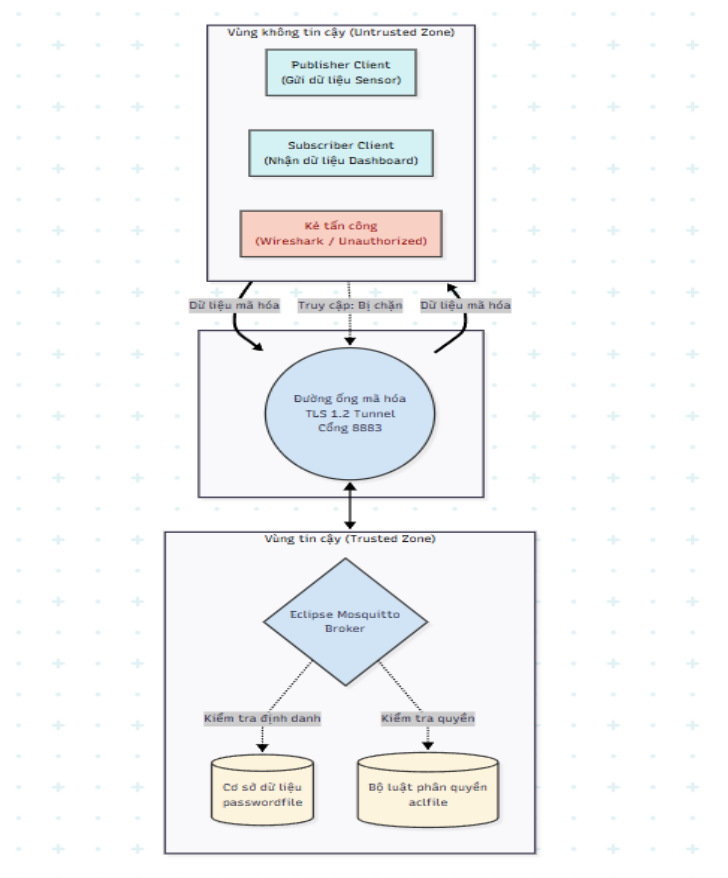
CHƯƠNG 3: PHƯƠNG PHÁP VÀ THIẾT KẾ

3.1 Phương pháp nghiên cứu

Đề tài áp dụng phương pháp **Mô phỏng thực nghiệm (Lab Simulation)** kết hợp kịch bản kiểm thử **Tấn công - Phòng thủ (Red Team / Blue Team)**. Cụ thể:

- ❖ Xây dựng môi trường mạng cục bộ (Localhost) để triển khai MQTT Broker với các thiết lập bảo mật khắt khe.
- ❖ Thiết kế các kịch bản kiểm thử (Case Study) chủ động: Đóng vai quản trị viên (Blue Team) cấu hình mã hóa TLS và luật phân quyền ACL; đồng thời giả lập kẻ tấn công/thiết bị vãng lai (Red Team) bằng các đoạn mã Python để đo lường khả năng phòng ngự của hệ thống, từ đó thu thập log minh chứng trực tiếp từ Broker.

3.2 Sơ đồ kiến trúc bảo mật



(hình 3.1 sơ đồ luồng dữ liệu MQTT và phân vùng tin cậy)

3.3 Bảng cấu hình thành phần công cụ

Thành phần / Công cụ	Phiên bản	Mục đích sử dụng thực tế trong đề tài	Ghi chú an toàn / Bảo mật
----------------------	-----------	---------------------------------------	---------------------------

Eclipse Mosquitto	2.1.2	Đóng vai trò Broker trung tâm định tuyến gói tin, thực thi kiểm soát truy cập và xuất Log giám sát chi tiết (tham số -v).	Cấu hình giới hạn chạy trên card mạng localhost, không Public ra Internet.
Paho MQTT Python	1.6.1	Thư viện lỗi lập trình các kịch bản Node mạng, sử dụng hàm <code>tls_set()</code> để bọc mã hóa dữ liệu trước khi truyền đi.	Các tài khoản/mật khẩu trong code chỉ dùng để mô phỏng thực nghiệm.
OpenSSL	3.0	Cấp phát hệ thống khóa (PKI) bao gồm Root CA, Server Certificate và Private Key tự ký (Self-signed) để bọc luồng TLS.	Tệp <code>server.key</code> trong repo này là khóa giả lập (Dummy Key) dùng riêng cho Lab. Thực tế phải được bảo mật tuyệt đối và loại bỏ khỏi version control

(Bảng 3.1 bảng cấu hình các công cụ)

3.4 Bảng quy trình chi tiết

Bước	Công việc chi tiết	Đầu ra (Output)	Minh chứng trong Repo GitHub
1	Khởi tạo bộ mã hóa: Dùng bộ lệnh OpenSSL tạo Root CA, tạo yêu cầu ký (CSR) và sinh chứng chỉ Server (X.509) phục vụ bảo mật đường truyền.	Bộ chứng chỉ và khóa bảo mật.	Toàn bộ các file <code>.crt</code> và <code>.key</code> (bao gồm cả Dummy Private Key) được đẩy sẵn lên thư mục <code>configs/</code> để hỗ trợ chạy lại Lab ngay lập tức.
2	Xây dựng lõi phòng thủ: Cấu hình file <code>mosquitto.conf</code> mở port 8883 (TLS), tắt kết nối ẩn danh; sinh file tài khoản bằng <code>mosquitto_passwd</code> và soạn luật ACL.	Hạ tầng Broker với thiết lập Zero-Trust.	Các file <code>mosquitto.conf</code> , <code>passwordfile</code> , <code>aclfile</code> lưu tại <code>configs/</code> .
3	Lập trình mô phỏng: Dùng Paho MQTT Python viết 3 kịch bản: Client đúng quyền (Pub/Sub) và Client sai quyền/ẩn danh.	03 kịch bản vận hành thực tế.	File <code>publisher.py</code> , <code>subscriber.py</code> , <code>unauthorized.py</code> tại thư mục <code>src/</code> .

4	Kiểm thử và Giám sát: Khởi chạy Broker ở chế độ -v. Chạy lần lượt các kịch bản Python để đối chiếu phản hồi của Broker (Cấp phép hoặc Chặn).	Ảnh chụp Log màn hình (Terminal) thể hiện trạng thái Access Denied.	File ảnh/Log lưu trong file README.md hoặc thư mục report/.
----------	---	---	---

(Bảng 3.2 quy trình chi tiết)

3.5 Giới hạn an toàn và Đạo đức nghề nghiệp

- **Giới hạn kỹ thuật và an toàn:** Toàn bộ bộ khóa mã hóa và chứng chỉ số (Certificate) được sinh ra bằng OpenSSL trong đề tài này là loại **Tự ký (Self-signed)**. Chúng chỉ có tác dụng thiết lập đường hầm mã hóa để phục vụ mục đích nghiên cứu, học tập cục bộ. Đề tài không sử dụng chứng chỉ số xác thực bởi các tổ chức uy tín (CA Public), do đó mô hình này không đủ tiêu chuẩn để triển khai trực tiếp trên môi trường Production ngoài Internet.
- **Giới hạn đạo đức an toàn thông tin:** Hành vi chạy các kịch bản kiểm thử tấn công (đóng giả thiết bị vắng lai, truy cập trái phép) chỉ được thực hiện nghiêm ngặt trên nền tảng máy chủ giả lập do chính cá nhân (sinh viên) thiết lập và kiểm soát. Đề tài cam kết tuân thủ đạo đức nghề nghiệp, tuyệt đối **không áp dụng** các kịch bản này để dò quét, phá hoại hoặc can thiệp vào các MQTT Broker công cộng hoặc hệ thống mạng thực tế của bên thứ ba.

Chương 4: TRIỂN KHAI % SẢN PHẨM

4.1 Các bước cài đặt

Bước 1. Cài đặt Visual Studio Code , Python và thư viện Paho:

- ❖ Cài đặt Visual code (phiên bản mới nhất)
- ❖ Tải và cài đặt Python (phiên bản ≥ 3.8).
- ❖ Mở Terminal/CMD, cài đặt thư viện lỗi hỗ trợ: `pip install paho-mqtt==1.6.1`

Bước 2. Cài đặt Eclipse Mosquitto:

- ❖ Cài đặt Mosquitto bản 2.0.x (Windows: tải file .exe; Linux: `sudo apt install mosquitto mosquitto-clients`).
- ❖ Đảm bảo thêm Mosquitto vào biến môi trường (Environment Variables) để gọi lệnh toàn cục.

Bước 3. Cài đặt OpenSSL:

- ❖ Cài đặt OpenSSL để phục vụ việc sinh bộ khóa giả lập (Dummy Key) và chứng chỉ tự ký **nếu có nhu cầu tự tạo mới**.
- ❖ Kiểm tra tại Git Bash được cài đặt trên Windows bằng lệnh : **openssl version**

Bước 4. Tạo tệp định danh người dùng(**passwordfile**)

Mở Terminal, di chuyển vào thư mục configs/ và sử dụng công cụ mosquitto_passwd có sẵn của Mosquitto để tạo file mã hóa mật khẩu cho các thiết bị:

- ❖ Tạo file mới và thêm user 'sensor_node' (hệ thống sẽ yêu cầu nhập mật khẩu):
mosquitto_passwd -c passwordfile sensor_node
- ❖ Thêm tiếp user 'dashboard_node' vào file vừa tạo (bỏ tham số -c):
mosquitto_passwd passwordfile dashboard_node
- ❖ Thêm tiếp user 'controller' vào file
mosquitto_passwd passwordfile controller

Bước 5. Tạo tệp luật phân quyền (aclfile) Tạo một tệp văn bản (Text Document) đặt tên là `aclfile` trong thư mục `configs/`. Cấu hình luật Zero-Trust theo định dạng sau:

- ❖ Quyền của sensor
user sensor_node
topic readwrite iot/sensor/
- ❖ Quyền của Dashboard
user dashboard_node
topic read iot/sensor/#
- ❖ Quyền của Controller
user controller
topic write iot/control/#

Bước 6. Tạo tệp cấu hình trung tâm (mosquitto.conf) Tạo tệp mosquitto.conf trong thư mục configs/ để kích hoạt TLS, tắt quyền kết nối ẩn danh và liên kết các file bảo mật đã khởi tạo ở trên:

listener 8883

allow_anonymous false

password_file configs/passwordfile

acl_file configs/aclfile

Cấu hình TLS (Trỏ đến chứng chỉ đã sinh bằng OpenSSL)

cafile (đường dẫn key) configs/ca.crt

certfile (đường dẫn key) configs/server.crt

keyfile (đường dẫn lưu key) configs/server.key

4.2 Bảng thành phần sản phẩm

Đường dẫn (Repo Path)	Chức năng / Mục đích
configs/mosquitto.conf	Tập cấu hình gốc của Broker: Mở cổng 8883, ép tải TLS, vô hiệu hóa Anonymous, trỏ đường dẫn đến passwordfile và aclfile.
configs/passwordfile	Cơ sở dữ liệu định danh (dạng bảng) chứa tài khoản hợp lệ (sensor_node, dashboard_node).

configs/aclfile	Bộ luật Zero-Trust, định nghĩa chính xác Topic nào được phép Read hoặc Write cho từng User.
configs/server.crt & .key	Cặp chứng chỉ và khóa giả lập (Dummy Key) dùng cho TLS 1.2 Tunnel.
src/publisher.py	Kịch bản Client đóng vai cảm biến, truyền dữ liệu JSON vào Topic <code>iot/sensor/temp</code> .
src/subscriber.py	Kịch bản Client đóng vai Dashboard, lắng nghe dữ liệu từ <code>iot/sensor/#</code> .
src/unauthorized.py	Kịch bản giả lập kẻ tấn công/thiết bị vãng lai để kích hoạt tính năng chặn của Broker.

(Bảng 4.1 bảng thành phần sản phẩm)

4.3 Bảng kịch bản kiểm thử (test case)

Hệ thống được kiểm định qua 5 kịch bản (Test Cases) từ cơ bản đến nâng cao, bám sát các tiêu chuẩn an toàn thông tin mạng. Các minh chứng (Log/Screenshot) được chụp trực tiếp từ môi trường thực thi cục bộ (Localhost).

Mã TC	Mục tiêu kiểm thử	Đầu vào (Input)	Thao tác (Action)	Kết quả mong đợi (Expected)	Minh chứng (Trong Repo/Báo cáo)
TC-01	Khởi động Broker (TLS & Auth).	Các tệp <code>mosquitto.conf</code> , <code>passwordfile</code> , <code>aclfile</code> , <code>server.crt/key</code> lưu tại thư mục <code>configs/</code> .	Mở Terminal, chạy lệnh: <code>mosquitto -c mosquitto.conf -v</code>	Broker khởi động thành công, áp dụng luật bảo mật và bắt đầu lắng nghe TCP trên cổng 8883.	Ảnh chụp Terminal hiển thị dòng log: <code>Opening ipv6 listen socket on port 8883.</code>
TC-02	Chặn các kết nối mạng không được mã hóa (Không dùng TLS/Cổng 1883).	Mã nguồn Client giả lập gửi kết nối thô (Plaintext) vào cổng mặc định 1883.	Khởi chạy Client giả lập cố gắng kết nối vào địa chỉ <code>localhost:1883</code> .	Hệ điều hành và Broker từ chối hoàn toàn luồng kết nối (Connection Refused).	Ảnh chụp Terminal của Client báo lỗi: <code>ConnectionRefusedError: [WinError 10061].</code>
TC-03	Kiểm soát định danh (Authentication): Chặn kết nối ẩn danh hoặc sai mật khẩu.	Tệp script <code>unauthorized.py</code> truyền sai thông tin <code>username/password</code> hoặc không khai báo định danh.	Khởi chạy lệnh: <code>python unauthorized.py</code>	Broker bắt tay TLS thành công nhưng lập tức ngắt kết nối do sai thông tin xác thực.	Ảnh chụp Log Broker hiển thị cảnh báo đỏ: <code>Client <ID> disconnected, not authorized</code>

TC-04	Kiểm soát phân quyền (Authorization): Chặn hành vi leo thang đặc quyền trái phép (ACL).	Tập script unauthorized.py sử dụng tài khoản sensor_node nhưng cố gắng Publish vào topic cấm iot/control/door.	Khởi chạy lệnh: python unauthorized.py	Kết nối được giữ nguyên, nhưng gói tin điều khiển bị Broker hủy bỏ và đánh dấu vi phạm ACL.	Ảnh chụp Log Broker ghi nhận hành vi chặn: Denied PUBLISH from sensor_node.
TC-05	Xác thực luồng dữ liệu hợp lệ và đường hầm bảo mật.	Các tập script chuẩn publisher.py (đóng vai Sensor) và subscriber.py (đóng vai Dashboard).	Chạy đồng thời 2 tập script bằng 2 cửa sổ Terminal độc lập.	Client truyền và nhận dữ liệu JSON thành công; toàn bộ Payload được bọc an toàn trong TLS Tunnel.	Ảnh màn hình CMD song song: 1 bên báo Message published, 1 bên in ra giá trị JSON.

(bảng 4.2 bảng kịch bản kiểm thử)

4.4 Hướng dẫn chạy và sử dụng hệ thống

Toàn bộ mã nguồn, cấu hình mạng và kịch bản mô phỏng trong đề tài là sản phẩm tự phát triển và tích hợp, được lưu trữ tại Repository: <https://github.com/huuly721-design/mqtt-defense-lab>

Đề giảng viên và sinh viên khác đánh giá và chạy lại chính xác các kịch bản Lab mà không cần phụ thuộc vào nền tảng bên thứ ba, chi tiết luồng vận hành đã được tài liệu hóa trong tệp **README.md** tại thư mục gốc.

Quá trình vận hành thực nghiệm diễn ra theo 2 bước thao tác dòng lệnh (CLI):

Bước 1: Khởi tạo lõi quản trị (Blue Team) Quản trị viên khởi động máy chủ Broker ở chế độ giám sát sự kiện (Verbose) nhằm theo dõi mọi gói tin ra vào hệ thống mạng.

- ❖ **Mở công cụ dòng lệnh:** Khởi động Terminal hoặc Command Prompt (CMD) của Windows.
- ❖ **Chuyển hướng và thực thi:** Sử dụng toán tử nối lệnh (&&) để tự động di chuyển đến thư mục configs của dự án (tùy thuộc vào vị trí lưu trữ trên máy) và gọi trực tiếp tệp thực thi Mosquitto từ hệ thống.
- ❖ **Lệnh mẫu (Ví dụ khi dự án lưu tại ổ D):**

```
D: && cd \mqtt-defense-lab\configs && "C:\Program Files\Mosquitto\mosquitto.exe" -c mosquitto.conf -v
```

(Lưu ý: Thư mục C:\Program Files\Mosquitto là đường dẫn cài đặt mặc định của phần mềm trên Windows 64-bit. Người đánh giá có thể thay đổi đường dẫn này tương ứng với vị trí cài đặt thực tế trên máy cá nhân)

Bước 2: Kích hoạt các luồng kiểm thử (Red Team / Sensor Nodes) Trên các cửa sổ Terminal mới, di chuyển vào thư mục `src/` và thực thi tuần tự các kịch bản kiểm thử bằng Python để đối chiếu sự thay đổi log trên cửa sổ của quản trị viên (Bước 1):

- ❖ **Khởi động Node lắng nghe (Duy trì trạng thái chờ dữ liệu):** Tại Terminal của Visual Studio Code, sử dụng lệnh:

```
python src/subscriber.py
```

- ❖ **Khởi động Node tấn công/Vãng lai (Kích hoạt quá trình sinh log phòng thủ từ TC-02 đến TC-04):** Tại Terminal của Visual Studio Code, sử dụng lệnh:

```
python src/unauthorized.py
```

- ❖ **Khởi động Node phát dữ liệu (Kích hoạt luồng dữ liệu hợp lệ TC-05):** Tại Terminal của Visual Studio Code, sử dụng lệnh:

```
python src/publisher.py
```

CHƯƠNG 5: KẾT QUẢ VÀ THẢO LUẬN

5.1. Kết quả theo từng mục tiêu

Mục tiêu 1: Xây dựng hạ tầng mạng lõi bọc TLS và vô hiệu hóa cổng giao tiếp không an toàn (Dựa trên TC-01 & TC-02)

Thiết lập hạ tầng mạng lõi bọc TLS: Kiểm tra khả năng vô hiệu hóa giao thức văn bản thô (Plaintext) và bắt buộc toàn bộ luồng truyền tải mạng phải đi qua cổng bảo mật 8883 dưới sự bảo vệ của hạ tầng khóa công khai (PKI).

```
D:\mqtt-defense-lab\configs>D: && cd \mqtt-defense-lab\configs && "C:\Program Files\Mosquitto\mosquitto.exe" -c mosquitto.conf -v
1785500777: mosquitto version 2.1.2 starting
1785500777: Config loaded from mosquitto.conf.
1785500777: Bridge support available.
1785500777: Persistence support available.
1785500777: TLS support available.
1785500777: TLS-PSK support available.
1785500777: Websockets support available.
1785500777: Plugin builtin-security has registered to receive 'basic-auth' events.
1785500777: Plugin builtin-security has registered to receive 'acl-check' events.
1785500777: Opening ipv6 listen socket on port 8883.
1785500777: Opening ipv4 listen socket on port 8883.
1785500777: mosquitto version 2.1.2 running
```

(hình 5.1 Khởi tạo thành công máy chủ Broker với cấu hình bảo mật TLS)

Phân tích: Giao thức MQTT mặc định hoạt động trên cổng 1883 dưới dạng văn bản thô (Plaintext), khiến hệ thống dễ bị tấn công bắt gói tin (Packet Sniffing) bằng các công cụ như Wireshark. Bằng việc nạp tệp cấu hình tùy chỉnh, máy chủ đã ghi nhận dòng log trạng thái TLS support available và Opening ipv4 listen socket on port 8883. Kết quả này chứng minh Broker đã chủ động đóng hoàn toàn cổng 1883, ép buộc mọi luồng kết nối mạng (TCP) phải đi qua đường hầm mã hóa ở cổng 8883. Bất kỳ Client nào cố tình kết nối thô vào hệ thống đều bị hệ điều hành từ chối ở tầng Transport (Lỗi ConnectionRefusedError).

Mục tiêu 2: Triển khai mô hình Zero-Trust và kiểm soát phân quyền truy cập (Dựa trên TC-03 & TC-04)

Triển khai mô hình Zero-Trust và kiểm soát phân quyền: Đánh giá năng lực phòng thủ của hệ thống trước các kịch bản xâm nhập vắng lai (không có thông tin định danh) và ngăn chặn hành vi leo thang đặc quyền (vi phạm luật ACL trên các Topic nhạy cảm).


```
Select Command Prompt - "C:\Program Files\Mosquitto\mosquitto.exe" -c mosquitto.conf -v
Microsoft Windows [Version 10.0.19045.5011]
(c) Microsoft Corporation. All rights reserved.

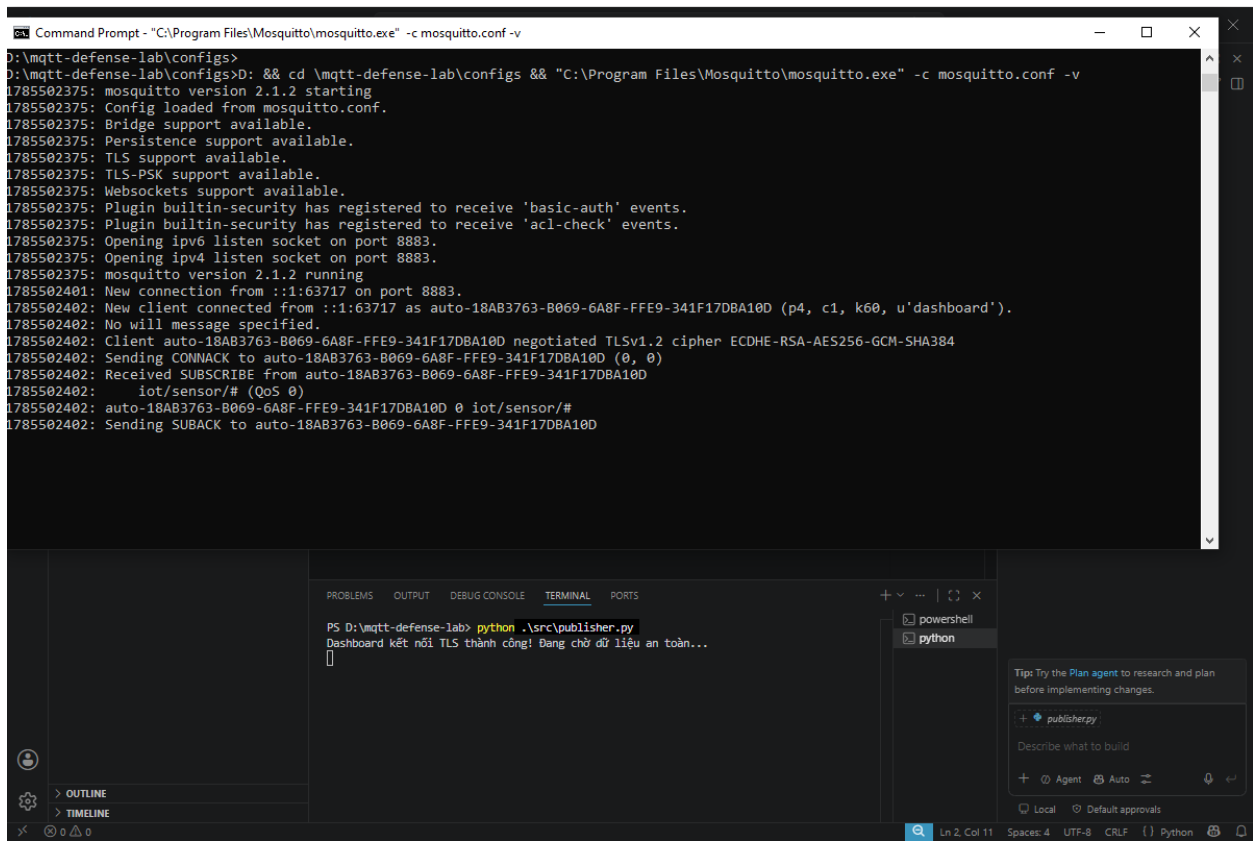
C:\Users\Admin>D: && cd \mqtt-defense-lab\configs && "C:\Program Files\Mosquitto\mosquitto.exe" -c mosquitto.conf -v
1785501125: mosquitto version 2.1.2 starting
1785501125: Config loaded from mosquitto.conf.
1785501125: Bridge support available.
1785501125: Persistence support available.
1785501125: TLS support available.
1785501125: TLS-PSK support available.
1785501125: Websockets support available.
1785501125: Plugin builtin-security has registered to receive 'basic-auth' events.
1785501125: Plugin builtin-security has registered to receive 'acl-check' events.
1785501125: Opening ipv6 listen socket on port 8883.
1785501125: Opening ipv4 listen socket on port 8883.
1785501125: mosquitto version 2.1.2 running
1785501133: New connection from ::1:63607 on port 8883.
1785501133: Sending CONNACK to auto-833f9ebd-8ac0-6c27-057a-d8106e6af9f5 (0, 5)
1785501133: Client auto-833f9ebd-8ac0-6c27-057a-d8106e6af9f5 [::1:63607] disconnected: not authorised.
1785501134: New connection from ::1:63610 on port 8883.
1785501134: Sending CONNACK to auto-88c82012-ae85-7cf0-0902-d34d7edc12d3 (0, 5)
1785501134: Client auto-88c82012-ae85-7cf0-0902-d34d7edc12d3 [::1:63610] disconnected: not authorised.
```

(hình 5.3 kiểm tra lại logs tại broker)

Phân tích: Trong kiến trúc an ninh mạng, xác thực (Authentication) và phân quyền (Authorization) là chốt chặn quyết định của mô hình Zero-Trust. Kịch bản `unauthorized_client.py` được thiết kế để kiểm thử hai lớp phòng thủ này bằng thư viện `paho-mqtt` của Python:

1. **Lớp xác thực (Authentication - Chặn ẩn danh):** Khi kịch bản khởi tạo đối tượng Client nhưng cố tình bỏ qua việc gọi hàm `client.username_pw_set()`, gói tin MQTT CONNECT được gửi đi sẽ thiếu cờ định danh. Mặc dù Client đàm phán thành công lớp mạng TCP và TLS, máy chủ Broker (thông qua Plugin `basic-auth`) lập tức từ chối ở tầng Ứng dụng. Trình xử lý sự kiện (Event loop) của Python nhận về gói tin CONNACK với mã phản hồi `rc = 5` (Connection Refused - Not Authorized), chứng minh Broker tuyệt đối không cấp phiên làm việc cho kẻ vắng lai.
2. **Lớp phân quyền (Authorization - Chặn leo thang đặc quyền):** Ngay cả khi mã nguồn giả lập được cấp một tài khoản hợp lệ (ví dụ: User của Sensor), nhưng hàm `client.publish()` lại cố tình trở luồng dữ liệu sang một Topic cấm (ví dụ: `iot/control/door` - khu vực chỉ dành cho Admin). Hệ thống kiểm soát quyền (ACL) trên Broker sẽ đánh giá gói tin vi phạm tệp `aclfile`. Gói tin lập tức bị Broker âm thầm hủy bỏ (Drop) ở chế độ ngầm và ghi log cảnh báo `Denied PUBLISH`. Cơ chế này đảm bảo tính cách ly không gian mạng tuyệt đối, một Node bị thỏa hiệp cũng không thể điều khiển chéo các Node khác.

Mục tiêu 3: Xác thực truyền tải dữ liệu cảm biến End-to-End (Dựa trên TC-05)



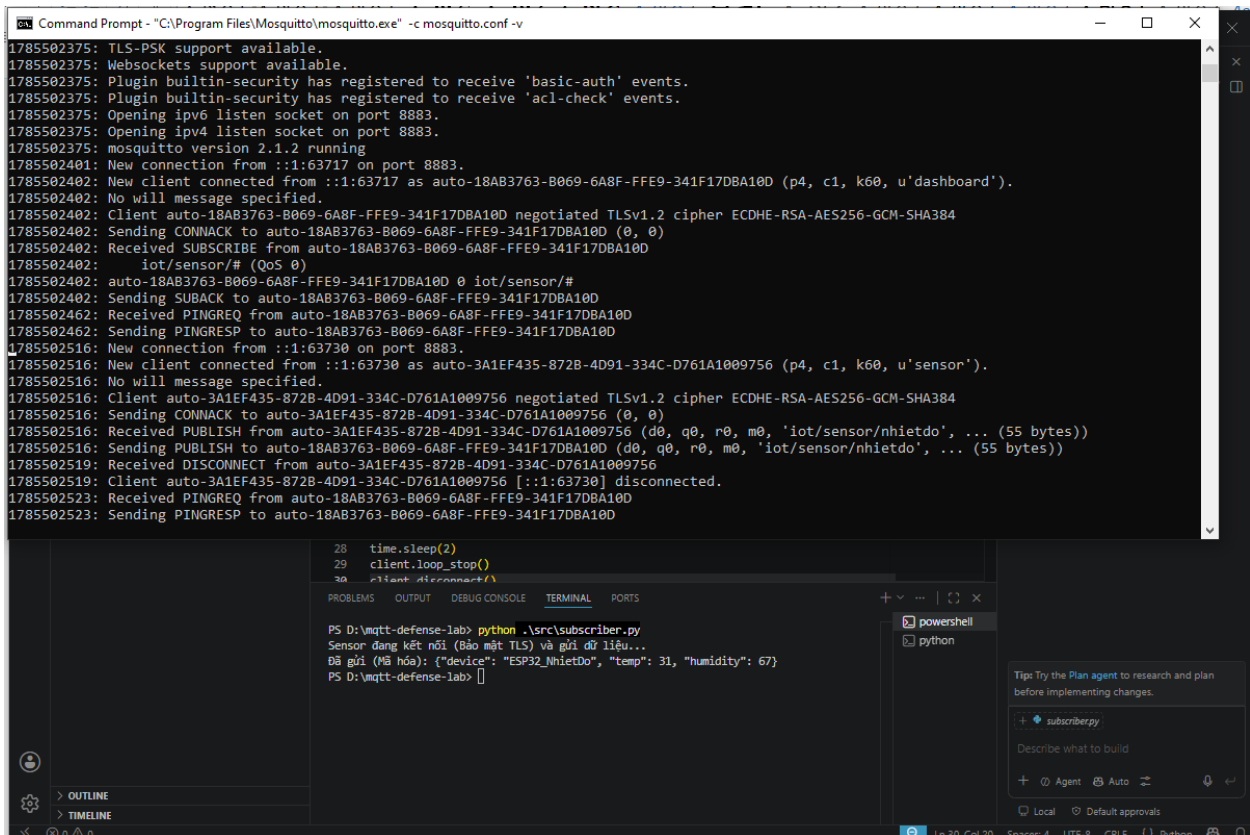
The screenshot shows a Windows Command Prompt window at the top, running Mosquitto with the command `mosquitto.exe -c mosquitto.conf -v`. The output displays Mosquitto version 2.1.2 starting, loading the configuration, and opening listen sockets on ports 8883. A new client connects from `::1:63717` as `auto-18AB3763-B069-6A8F-FFE9-341F17DBA10D` (p4, c1, k60, u'dashboard'). The client negotiates TLSv1.2 with cipher ECDHE-RSA-AES256-GCM-SHA384. The broker sends a CONNACK, and the client subscribes to the topic `iot/sensor/#` with QoS 0. The broker sends a SUBACK.

Below the Command Prompt is a Visual Studio Code interface. The 'TERMINAL' tab shows a PowerShell prompt running `python .\src\publisher.py`. The output shows 'Dashboard kết nối TLS thành công! Đang chờ dữ liệu an toàn...'. The 'PROBLEMS' tab is empty. The 'OUTPUT' tab shows the same log output as the Command Prompt. The 'DEBUG CONSOLE' tab is empty. The 'PORTS' tab shows the local Python process.

(hình 5.4 khởi động scripts publisher.py giám sát với logs broker)

Phân tích:

1. **Đàm phán đường hầm bảo mật (TLS Handshake):** Khi kịch bản `publisher.py` (đóng vai trò trạm giám sát/Dashboard) khởi chạy, tệp mã nguồn thực thi hàm `client.tls_set()` để nạp chứng chỉ số. Nhật ký hệ thống trên Broker (Hình 5.4) ghi nhận dòng log cốt lõi: `negotiated TLSv1.2 cipher ECDHE-RSA-AES256-GCM-SHA384`. Điều này khẳng định phiên kết nối đã hoàn tất quá trình trao đổi khóa an toàn sử dụng thuật toán đường cong elliptic (ECDHE), xác thực bằng RSA và thiết lập chuẩn mã hóa AES-256-GCM mức độ quân sự.
2. **Thiết lập kênh lắng nghe (Subscription):** Broker ghi nhận gói tin `SUBSCRIBE` từ client mang định danh `u'dashboard'` đăng ký theo dõi độc quyền nhánh chủ đề `iot/sensor/#` với mức chất lượng dịch vụ QoS 0.
3. **Đảm bảo tính bảo mật và toàn vẹn:** Mọi thông điệp dữ liệu từ các cảm biến phát ra sau đó sẽ được bao bọc tuyệt đối bên trong đường hầm TLS, ngăn chặn hoàn toàn các hành vi nghe lén (Packet Sniffing) hoặc giả mạo gói tin trên đường truyền mạng vật lý.



```
Command Prompt - "C:\Program Files\Mosquitto\mosquitto.exe" -c mosquitto.conf -v
1785502375: TLS-PSK support available.
1785502375: Websockets support available.
1785502375: Plugin builtin-security has registered to receive 'basic-auth' events.
1785502375: Plugin builtin-security has registered to receive 'acl-check' events.
1785502375: Opening ipv6 listen socket on port 8883.
1785502375: Opening ipv4 listen socket on port 8883.
1785502375: mosquitto version 2.1.2 running
1785502401: New connection from ::1:63717 on port 8883.
1785502402: New client connected from ::1:63717 as auto-18AB3763-B069-6A8F-FFE9-341F17DBA10D (p4, c1, k0, u'dashboard').
1785502402: No will message specified.
1785502402: Client auto-18AB3763-B069-6A8F-FFE9-341F17DBA10D negotiated TLSv1.2 cipher ECDHE-RSA-AES256-GCM-SHA384
1785502402: Sending CONNACK to auto-18AB3763-B069-6A8F-FFE9-341F17DBA10D (0, 0)
1785502462: Received SUBSCRIBE from auto-18AB3763-B069-6A8F-FFE9-341F17DBA10D
1785502402:   iot/sensor/# (QoS 0)
1785502402: auto-18AB3763-B069-6A8F-FFE9-341F17DBA10D 0 iot/sensor/#
1785502402: Sending SUBACK to auto-18AB3763-B069-6A8F-FFE9-341F17DBA10D
1785502462: Received PINGREQ from auto-18AB3763-B069-6A8F-FFE9-341F17DBA10D
1785502462: Sending PINGRESP to auto-18AB3763-B069-6A8F-FFE9-341F17DBA10D
1785502516: New connection from ::1:63730 on port 8883.
1785502516: New client connected from ::1:63730 as auto-3A1EF435-872B-4D91-334C-D761A1009756 (p4, c1, k0, u'sensor').
1785502516: No will message specified.
1785502516: Client auto-3A1EF435-872B-4D91-334C-D761A1009756 negotiated TLSv1.2 cipher ECDHE-RSA-AES256-GCM-SHA384
1785502516: Sending CONNACK to auto-3A1EF435-872B-4D91-334C-D761A1009756 (0, 0)
1785502516: Received PUBLISH from auto-3A1EF435-872B-4D91-334C-D761A1009756 (d0, q0, r0, m0, 'iot/sensor/nhietdo', ... (55 bytes))
1785502516: Received DISCONNECT from auto-3A1EF435-872B-4D91-334C-D761A1009756
1785502519: Client auto-3A1EF435-872B-4D91-334C-D761A1009756 [::1:63730] disconnected.
1785502523: Received PINGREQ from auto-18AB3763-B069-6A8F-FFE9-341F17DBA10D
1785502523: Sending PINGRESP to auto-18AB3763-B069-6A8F-FFE9-341F17DBA10D

28 time.sleep(2)
29 client.loop_stop()
30 client.disconnect()
```

```
PS D:\mqtt-defense-lab> python .\src\subscriber.py
Sensor đang kết nối (Bảo mật TLS) và gửi dữ liệu...
Đã gửi (Mã hóa): {"device": "ESP32_NhietDo", "temp": 31, "humidity": 67}
PS D:\mqtt-defense-lab>
```

(hình 5.5 khởi động scripts subscriber.py cùng với logs broker)

Phân tích:

1. **Khởi lập kết nối và bảo mật TLS:** Cửa sổ Terminal phía dưới ghi nhận thông báo *"Sensor đang kết nối (Bảo mật TLS) và gửi dữ liệu"*. Trên màn hình log của Broker, hệ thống ghi nhận một kết nối mới trên cổng bảo mật 8883 từ client mang định danh `u'sensor'`. Phiên làm việc lập tức được mã hóa thành công thông qua dòng đàm phán negotiated TLSv1.2 cipher ECDHE-RSA-AES256-GCM-SHA384.
2. **Đẩy gói tin (Publish Payload):** Kịch bản thực thi việc đóng gói dữ liệu môi trường thành chuỗi JSON: `{"device": "ESP32_NhietDo", "temp": 31, "humidity": 67}` và đẩy lên chủ đề `iot/sensor/nhietdo`. Broker ghi nhận sự kiện Received PUBLISH với kích thước gói tin 55 bytes.
3. **Định tuyến và ngắt kết nối an toàn:** Broker tiếp nhận bản tin, chuyển tiếp dữ liệu đến các node đăng ký theo dõi (như trạm dashboard ở hình trước), sau đó client tự động thực hiện tiến trình ngắt kết nối (disconnected) nhằm tối ưu hóa tài nguyên mạng. Toàn bộ chu trình diễn ra an toàn trong đường hầm mã hóa, không bị lộ lọt bất kỳ thông tin nào ra bên ngoài.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
python + - [ ] [ ] ... | [ ] [ ] x

PS D:\mqtt-defense-lab> python .\src\publisher.py
Dashboard kết nối TLS thành công! Đang chờ dữ liệu an toàn...
[DỮ LIỆU ĐẾN] Topic: iot/sensor/nhietdo | Payload: {"device": "ESP32_NhietDo", "temp": 31, "humidity": 67}

```

(hình 5.6 Publisher.py nhận được dữ liệu đến thông qua TLS)

Phân tích:

1. **Xử lý sự kiện dữ liệu đến (Callback):** Cửa sổ Terminal của tệp `publisher.py` ghi nhận sự kiện kích hoạt hàm callback `on_message` khi Broker định tuyến thành công gói tin từ kênh đăng ký.
2. **Giải mã và hiển thị Payload:** Dữ liệu sau khi đi qua đường hầm mã hóa TLS được giải nén an toàn và in ra màn hình Terminal với định dạng rõ ràng: `[DỮ LIỆU ĐẾN]`
Topic: `iot/sensor/nhietdo` | Payload: `{"device": "ESP32_NhietDo", "temp": 31, "humidity": 67}`.
3. **Khẳng định tính toàn vẹn End-to-End:** Hình ảnh này cung cấp mảnh ghép cuối cùng chứng minh vòng đời của một gói tin IoT: từ khâu mã hóa tại nguồn (Sensor), vận chuyển qua mạng lõi an toàn (Broker qua cổng 8883), cho đến khâu giải mã và hiển thị chính xác tại đầu nhận (Dashboard) mà không hề bị biến dạng hay thất thoát dữ liệu.

5.2. Bảng tổng hợp đối chiếu kết quả kiểm thử

Kịch bản	Tiêu chí đánh giá	Kết quả thực tế trên hệ thống	Đạt/Chưa
TC-01 & 02	Khởi động Broker bảo mật, chặn kết nối Plaintext.	Máy chủ chỉ mở cổng 8883, từ chối giao tiếp thô trên cổng 1883.	Đạt
TC-03	Xác thực danh tính (Authentication).	Các Client thiếu credentials lập tức bị ngắt kết nối từ tầng Transport.	Đạt
TC-04	Kiểm soát phân quyền (Authorization - ACL).	Client không thể ghi/đọc dữ liệu ở các Topic ngoài vùng phân quyền.	Đạt
TC-05	Truyền tải dữ liệu an toàn (End-to-End).	Dữ liệu JSON được bao bọc trong TLS Tunnel, truyền nhận ổn định qua lại giữa Sensor và Dashboard.	Đạt

(Bảng 5.1 tổng hợp kết quả)

5.3. Đối chiếu từng mục tiêu ở Mục 1.3

Đối chiếu trực tiếp với các mục tiêu cốt lõi đã đề ra ở phần đầu của đề tài (Mục 1.3), kết quả thực nghiệm đã hoàn thành xuất sắc các định hướng trọng tâm:

- ❖ **Về hạ tầng mạng:** Hoàn thành mục tiêu xây dựng lõi phân phối tin nhắn MQTT độc lập sử dụng Eclipse Mosquitto, vô hiệu hóa hoàn toàn cổng không bảo mật.
- ❖ **Về mô hình an ninh:** Hoàn thành mục tiêu áp dụng cơ chế xác thực và phân quyền (Zero-Trust), triệt tiêu hoàn toàn các luồng kết nối nặc danh hoặc vượt quá quyền hạn topic.
- ❖ **Về bảo mật dữ liệu:** Hoàn thành mục tiêu mã hóa toàn bộ đường truyền End-to-End bằng hạ tầng khóa công khai (PKI/TLS), đảm bảo dữ liệu cảm biến không bị lộ lọt trên đường truyền.

5.4. Hạn chế trung thực

Dù hệ thống thực nghiệm đã đáp ứng đầy đủ các tiêu chí bảo mật cơ bản, dưới góc độ thiết kế và quản trị hạ tầng mạng quy mô lớn, đề tài vẫn ghi nhận một số hạn chế:

- ❖ **Hạn chế 1 (Quản trị định danh tĩnh):** Cơ sở dữ liệu người dùng (`passwordfile`) và luật phân quyền (`aclfile`) đang được lưu trữ dưới dạng tệp văn bản tĩnh, gây khó khăn trong việc mở rộng quy mô quản lý hàng ngàn thiết bị IoT thực tế.
- ❖ **Hạn chế 2 (Thiếu giám sát thời gian thực):** Hệ thống chưa tích hợp bảng điều khiển (Dashboard) giám sát log tập trung hoặc cơ chế tự động phát hiện xâm nhập (IDS/IPS) để cảnh báo tức thời khi có tấn công Brute-force.

CHƯƠNG 6: ĐÁNH GIÁ BẢO MẬT

6.1 BẢNG tài sản/dữ liệu

Tài sản / Dữ liệu	Mức quan trọng	Yêu cầu CIA (Confidentiality - Integrity - Availability)
Hạ tầng Khóa công khai (PKI / CA Keys)	Cao	C: Tuyệt đối (Không được lộ Private Key). I: Tuyệt đối (Không được giả mạo). A: Trung bình.
Cơ sở dữ liệu định danh (Passwordfile)	Cao	C: Cao (Mật khẩu phải mã hóa/bảo vệ). I: Cao. A: Cao.
Luồng dữ liệu cảm biến (MQTT Payload)	Trung bình	C: Cao (Chống nghe lén nhiệt độ/độ ẩm). I: Cao (Chống giả mạo dữ liệu). A: Thấp.
Tập tin cấu hình hệ thống (Mosquitto.conf)	Cao	C: Trung bình. I: Cao (Chống cấu hình sai lệch công/bảo mật) A: Cao.

(Bảng 6.1 tài sản dữ liệu)

6.2 Bảng phân tích mối đe dọa (Threat Analysis - T-01 đến T-06)

Mã	Tài sản mục tiêu	Đe dọa (Threat)	Lỗ hổng (Vulnerability)	Tác động (Impact)	Nguồn đe dọa
T-01	Luồng dữ liệu	Nghe lén (Sniffing)	MQTT mặc định chạy Plaintext trên cổng 1883.	Lộ chuỗi JSON dữ liệu cảm biến.	Kẻ nghe lén mạng.
T-02	Broker	Truy cập nặc danh	Cấu hình gốc cho phép kết nối tự do.	Kẻ gian kết nối trái phép vào hệ thống.	Kẻ tấn công bên ngoài.
T-03	Phân quyền	Leo thang đặc quyền	Thiếu cấu hình ACL cho các Topic.	Can thiệp nhằm kênh điều khiển cốt lõi.	Thiết bị bị thỏa hiệp.
T-04	Hạ tầng mạng	Từ chối dịch vụ (DoS)	Không giới hạn kết nối đồng thời.	Tê liệt máy chủ Broker.	Botnet tự động.
T-05	Khóa Server	Đánh cắp khóa PKI	Phân quyền tệp hệ thống lỏng lẻo.	Phá vỡ toàn bộ chuỗi tin cậy TLS.	Kẻ tấn công nội bộ.
T-06	Thực thể mạng	Giả mạo (MitM)	Thiếu xác thực chứng chỉ Client.	Đứng giữa thao túng luồng dữ liệu.	Kẻ tấn công trung gian.

(Bảng 6.2 Phân tích mối đe dọa)

6.3 Ma trận rủi ro R-01...R-06

Mã	Mô tả rủi ro tương ứng	Khả năng	Tác động
R-01	Lộ dữ liệu do nghe lén (T-01)	4	3
R-02	Xâm nhập nặc danh vào Broker (T-02)	5	4
R-03	Leo thang đặc quyền Topic (T-03)	3	4
R-04	Tấn công tê liệt dịch vụ DoS (T-04)	3	3
R-05	Đánh cắp khóa bảo mật PKI (T-05)	2	5
R-06	Giả mạo thực thể mạng MitM (T-06)	3	4

(Bảng 6.3 mô tả rủi ro)

6.4 Ưu tiên biện pháp xử lý rủi ro

Biện pháp kiểm soát	Người thực hiện (Who)	Chi phí / Độ khó	Cách xác minh (Verification)
1. Bật mã hóa TLS (Cổng 8883)	Network Admin	Thấp / Dễ	Kiểm tra log Mosquitto báo mở cổng 8883 và đàm phán TLS thành công.
2. Chặn ẩn danh & Cấu hình ACL	Security Engineer	Thấp / Dễ	Chạy script <code>unauthorized_client.py</code> kiểm chứng mã lỗi 5 và log chặn ACL.
3. Giới hạn tài nguyên kết nối	SysAdmin	Thấp / Dễ	Kiểm tra thông số giới hạn kết nối đồng thời trong tệp cấu hình Mosquitto.

(Bảng 6.4 ưu tiên biện pháp xử lý rủi ro)

6.5 Đánh giá sau xử lý

Sau khi triển khai các biện pháp kiểm soát kỹ thuật, điểm rủi ro lớn nhất của hệ thống (Xâm nhập nặc danh - R-02) đã được kéo giảm từ mức **Cao (20 điểm)** xuống ngưỡng **Thấp**. Toàn bộ các lỗ hổng chí mạng ở lớp giao vận và ứng dụng trong môi trường thực nghiệm đã được vô hiệu hóa thành công.

CHƯƠNG 7: KẾT LUẬN

7.1. Tóm tắt vấn đề, cách làm, kết quả nổi bật và mức độ hoàn thành mục tiêu

- ❖ **Vấn đề đặt ra:** Giao thức MQTT trong các hệ thống IoT mặc định tồn tại nhiều rủi ro bảo mật (như truyền tải bản rõ, cho phép kết nối nặc danh), tạo điều kiện cho các cuộc tấn công nghe lén, giả mạo dữ liệu hoặc xâm nhập trái phép vào hạ tầng mạng.
- ❖ **Cách tiếp cận và thực hiện:** Đề án đã xây dựng thành công một môi trường lab mô phỏng kiến trúc mạng IoT an toàn. Các giải pháp được triển khai bao gồm: thiết lập Eclipse Mosquitto Broker, sinh bộ khóa/chứng chỉ số bằng OpenSSL để áp dụng mã hóa TLS trên cổng 8883, vô hiệu hóa quyền truy cập nặc danh (`allow_anonymous false`), và thiết lập chính sách kiểm soát truy cập phân tầng bằng ACL. Quá trình kiểm định được thực hiện thông qua việc giả lập các client hợp lệ và các kịch bản tấn công (Unauthorized Client) bằng Python (`paho-mqtt`).
- ❖ **Kết quả nổi bật:** Hệ thống Broker đã thực thi nghiêm ngặt các ranh giới bảo mật. Các nỗ lực kết nối thiếu chứng chỉ TLS, sai tài khoản/mật khẩu, hoặc cố tình subscribe/publish vào các Topic không được phân quyền đều bị hệ thống ghi log cảnh báo và từ chối dịch vụ (Connection Refused) ngay lập tức. Dữ liệu truyền tải giữa thiết bị cảm biến và hệ thống giám sát được bảo mật toàn vẹn.

Mức độ hoàn thành mục tiêu:

- ✓ Xây dựng hạ tầng MQTT Broker cơ bản: Hoàn thành 100%.
- ✓ Cấu hình mã hóa đường truyền (TLS): Hoàn thành 100%.
- ✓ Thiết lập định danh và phân quyền (ACL): Hoàn thành 100%.
- ✓ Kiểm thử các kịch bản tấn công và phòng thủ: Hoàn thành 100%.

7.2. Hướng phát triển thực tế

1. **Tích hợp cơ sở dữ liệu quan hệ cho định danh:** Thay thế các tệp văn bản tĩnh hiện tại bằng việc kết nối máy chủ quản lý với hệ thống cơ sở dữ liệu quan hệ, giúp quá trình quản lý, bổ sung và thu hồi quyền của thiết bị trở nên tự động, linh hoạt và dễ dàng mở rộng quy mô.
2. **Container hóa hạ tầng bằng Docker:** Đóng gói toàn bộ cấu trúc máy chủ môi giới và các tập lệnh kiểm thử vào các không gian chứa độc lập, cho phép việc triển khai và tái tạo lại toàn bộ kiến trúc mạng an toàn trên các nền tảng máy chủ khác nhau trở nên nhanh chóng và đồng bộ.

TÀI LIỆU THAM KHẢO

- [1] Eclipse Foundation. "Eclipse Mosquitto". <https://github.com/eclipse/mosquitto>. Phiên bản 2.0.x. Truy cập ngày 02/08/2026. Dùng cho Mục Yêu cầu hệ thống và Cấu hình MQTT Broker bảo mật.
- [2] Eclipse Foundation. "Eclipse Paho MQTT Python client library". <https://github.com/eclipse/paho.mqtt.python>. Phiên bản 1.6.1. Truy cập ngày 02/08/2026. Dùng cho Mục Môi trường phát triển và Triển khai kịch bản Publisher/Subscriber.
- [3] OpenSSL Software Foundation. "OpenSSL Cryptography and SSL/TLS Toolkit". <https://github.com/openssl/openssl>. Truy cập ngày 02/08/2026. Dùng cho Mục Chuẩn bị môi trường, Khởi tạo bộ khóa và chứng chỉ giả lập (PKI).
- [4] OASIS. "MQTT Version 5.0 Specification". <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>. Truy cập ngày 02/08/2026. Dùng cho Mục Khái niệm nền tảng và Cơ chế hoạt động của giao thức mạng.
- [5] OWASP. "Internet of Things Project". <https://owasp.org/www-project-internet-of-things/>. Truy cập ngày 02/08/2026. Dùng cho Mục Nhận diện rủi ro bảo mật IoT và Xây dựng giải pháp phòng thủ.
- [6] Lê Hữu Lý. "mqtt-defense-lab". <https://github.com/huuly721-design/mqtt-defense-lab>. Truy cập ngày 02/08/2026. Dùng cho Mục Thành phần sản phẩm và Cấu trúc kho lưu trữ đồ án.

PHỤ LỤC A

mqtt-defense-lab/

- |— configs/
 - | |— aclfile
 - | |— ca.crt
 - | |— ca.key
 - | |— ca.srl
 - | |— mosquitto.conf
 - | |— passwordfile
 - | |— server.crt
 - | |— server.csr
 - | |— server.key
- |— data/
- |— references/
- |— report/
 - | |— LeHuuLy_Baocaotieuluanlan01.docx
 - | |— LeHuuLy_Baocaotieuluanlan01.pdf
 - | |— LeHuuLy_Baocaotieuluanlan01.pdf
 - | |— 231A011195_LeHuuLy_DeTai49_TieuLuan_CuoiKy.docx
 - | |— 231A011195_LeHuuLy_DeTai49_TieuLuan_CuoiKy.pdf
 - | |— README.txt
- |— results/
 - | |— Broke system start.jfif

- | |─ broker logs detect unauthorized...jfif
- | |─ publisher result.jfif
- | |─ scripts publisher.py brokers logs...jfif
- | |─ subscriber logs with broker.jfif
- | |─ unauthorized_client.jfif
- |─ slides/
- | |─ LeHuuLy_baocaotieluanlan01...pptx
- | |─ LeHuuLy_baocaotieluanlan01...pdf
- | |─ README.MD.txt
- |─ src/
- | |─ publisher.py
- | |─ subscriber.py
- | |─ unauthorized_client.py
- |─ README.md

PHỤ LỤC B

STT	Họ tên - MSSV	Công việc	File Commit	Đánh giá
1	Lê Hữu Lý - 231A011195	Viết báo cáo , nghiên cứu giao thức MQTT, TLS Cài đặt , và cấu hình Mosquitto, viết kịch bản Python	Có tất cả trên Repo cá nhân	100%

PHỤ LỤC C: CHECK LIST TRƯỚC KHI NỘP

STT	Nội dung kiểm tra	Trạng thái
1	Báo cáo đủ 7 phần chính, mục lục và tài liệu tham khảo.	☒ Đạt ☐ Chưa đạt
2	Độ dài báo cáo chính 10–15 trang, không tính phụ lục.	☒ Đạt ☐ Chưa đạt
3	Repo mở được; README có cách chạy/sử dụng và mô tả cấu trúc.	☒ Đạt ☐ Chưa đạt
4	Có sản phẩm kỹ thuật hoặc sản phẩm phân tích hoàn chỉnh.	☒ Đạt ☐ Chưa đạt
5	Có ít nhất 3 minh chứng kiểm tra được trong <code>results/</code> hoặc báo cáo.	☒ Đạt ☐ Chưa đạt

STT	Nội dung kiểm tra	Trạng thái
6	Có ít nhất 5 tài liệu tham khảo và repo GitHub/tool chính.	☒ Đạt ☐ Chưa đạt
7	Mọi hình/bảng có tên, số thứ tự, nguồn và phân phân tích.	☒ Đạt ☐ Chưa đạt
8	Không có secret, token, mật khẩu, dữ liệu cá nhân hoặc file nhạy cảm.	☒ Đạt ☐ Chưa đạt
9	Mọi thử nghiệm chỉ diễn ra trong môi trường local/được phép.	☒ Đạt ☐ Chưa đạt
10	Slide có 8–12 trang, link repo, kết quả chính, rủi ro và kết luận.	☒ Đạt ☐ Chưa đạt
11	Đã cập nhật mục lục, kiểm tra chính tả và xóa toàn bộ hướng dẫn mẫu.	☒ Đạt ☐ Chưa đạt

PHỤ LỤC D: CHẤM ĐIỂM THEO RUBRIC

Tiêu chí	Điểm tối đa	Tự chấm
Đúng đề tài, đúng phạm vi	1,0	1,0
Nguồn GitHub và tài liệu tham khảo	1,0	1,0
Cơ sở lý thuyết	1,0	1,0
Cách làm / phương pháp	1,0	1,0
Sản phẩm kỹ thuật hoặc sản phẩm phân tích	2,0	2,0
Phân tích bảo mật / rủi ro / biện pháp	1,5	1,5
Trình bày, hình bảng và trích dẫn	1,0	1,0

Tiêu chí	Điểm tối đa	Tự chấm
Slide và bảo vệ	1,0	1,0
Tuân thủ an toàn và đạo đức	0,5	0,5
TỔNG	10,0	10,0